FORMAL PROJECT REPORT

Multi-Vendor Marketplace Platform

Course: Full stack .Net Web Developer

Technology Stack: Clean Architecture + ASP.NET Core MVC, Tailwind CSS, SQL Server + Entity Framework Core, Stripe API

Submission Deadline: Due: Friday, 17 July 2026, 11:59 PM

Table of Contents

1. Project Planning & Management
2. Literature Review
3. Requirements Gathering

1. Project Planning & Management

1.1 Project Proposal

The "Multi-Vendor Marketplace Platform" is a centralized e-commerce ecosystem designed to facilitate transactions between multiple independent sellers and global customers. Unlike traditional single-vendor stores, this platform serves as a middleman, providing the infrastructure for product listing, secure payments, and order fulfillment tracking [1.5].

Objectives:

- **Scalability:** Utilizing **Clean Architecture + ASP.NET Core MVC** paired with **SQL Server** and Entity Framework Core to ensure the system can handle a growing number of vendors and high-traffic volumes without performance degradation. The frontend is built with **Tailwind CSS** for a consistent, utility-first responsive UI [1.8].
- **Commission Management:** Automating the calculation and distribution of platform fees and vendor payouts.
- **Secure Payments:** Implementing Stripe API to facilitate PCI-compliant transactions and split-payment functionality [1.4].

Scope (MVP): The Minimum Viable Product focuses on core functionalities: Vendor/Customer Dashboards, Product CRUD (Create, Read, Update, Delete), Shopping Cart, Secure Checkout via Stripe Sandbox, and Order Tracking.

1.2 Project Plan (11-Week Timeline)

The project follows a structured 11-week development lifecycle, commencing on April 19, 2026, and concluding with final deployment on July 17, 2026. This timeline ensures adequate time for complex integrations such as the Stripe Connect gateway [1.4].

PHASE	WEEK	START - END DATE	KEY DELIVERABLES/MILESTONES
Discovery	1 - 2	Apr 19 - May 02	**Milestone:** Market research, project setup, and database schema design.
Design	3 - 4	May 03 - May 16	Wireframes, UI/UX prototyping (ASP.NET Core MVC views styled with Tailwind CSS), and API documentation.
Implementation (I)	5 - 6	May 17 - May 30	Vendor/Customer authentication (Identity Framework) and Product CRUD.
Implementation (II)	7 - 8	May 31 - Jun 13	Cart logic, Order Management, and Commission Calculation Engine.
Payment Integration	9	Jun 14 - Jun 20	**Milestone:** Stripe API sandbox integration and webhook setup.
Testing & QA	10	Jun 21 - Jun 27	Unit testing, UAT (User Acceptance Testing), and bug fixing.
Deployment	11	Jun 28 - Jul 17	**Final Milestone:** Production deployment and final presentation.

1.3 Task Assignment & Roles

ROLE	PRIMARY RESPONSIBILITIES
Project Manager	Strategic planning, resource allocation, timeline oversight, and stakeholder communication [1.3].
ASP.NET Backend Developer	Core business logic, Entity Framework Core migrations, API development, and database security.
Frontend Developer	Developing responsive Tailwind CSS-styled ASP.NET Core MVC views, UI components, and client-side validation.
QA Tester	End-to-end testing, Stripe Sandbox transaction verification, and performance benchmarking.

1.4 Risk Assessment & Mitigation Plan

Building a multi-vendor platform involves significant technical and financial risks. Proactive mitigation is essential for system integrity [1.2].

- **Payment Failures:** Risk of transaction timeouts or API errors. *Mitigation:* Rigorous Stripe sandbox testing and implementing robust error-handling/logging middleware.
- **Vendor Data Breaches:** Risk of sensitive seller data exposure. *Mitigation:* Utilizing ASP.NET Identity Framework with MFA (Multi-Factor Authentication) and data encryption at rest.
- **Project Delays:** Risk of missing the July 17 deadline. *Mitigation:* Bi-weekly sprints and prioritized backlog management to ensure MVP features are completed first.

1.5 Key Performance Indicators (KPIs)

To measure project success, the following technical and operational metrics will be tracked [1.2]:

1. **System Uptime:** Target of 99.9% availability through cloud hosting and load balancing.

2. **API Response Time:** Average latency of <200ms for core product listing requests.
3. **User Adoption Rate:** Number of active vendors and customer sign-ups within the first month post-launch.
4. **Successful Transaction Rate:** Percentage of completed checkouts vs. abandoned carts.

2. Literature Review

2.1 Industry Analysis: Etsy vs. Amazon Marketplace

The multi-vendor landscape is dominated by two primary models: niche-community and mass-scale logistics [1.1].

FEATURE	ETSY MODEL	AMAZON MODEL
Target Audience	Creative entrepreneurs and vintage enthusiasts.	General consumers seeking convenience/speed.
Seller Focus	Personalized branding, community engagement [1.9].	High volume, fulfillment-heavy (FBA).
Commission Structure	Listing fees + fixed transaction percentage.	Referral fees + subscription fees + fulfillment costs.

2.2 Suggested Improvements & Innovation

Based on current industry trends, our platform will incorporate several enhancements beyond standard marketplace functionality:

- **Advanced Vendor Analytics:** Implementing data visualization tools (e.g., Chart.js) within the vendor dashboard to track sales trends and customer demographics [1.2].
- **Mobile-First Responsive Design:** Leveraging **Tailwind CSS** utility classes to ensure full, consistent functionality across all screen sizes, catering to the increasing percentage of mobile shoppers [1.8].

- **Optimized Commission Logic:** Using ASP.NET Core background services to calculate tiered commissions dynamically based on vendor performance or product category.

2.3 Final Evaluation & Grading Criteria

The project will be assessed based on the following weighted criteria to ensure a balanced focus on theory and practice:

- **Documentation (25%):** Quality of the project report, requirements clarity, and architecture diagrams.
- **Implementation/Code (40%):** Cleanliness of C# code, efficient EF Core queries, and overall system functionality.
- **Testing/QA (20%):** Evidence of unit testing, security audits, and Stripe integration stability.
- **Presentation (15%):** Ability to demonstrate the system effectively and respond to technical inquiries.

3. Requirements Gathering

3.1 Stakeholder Analysis

Identifying the needs of all parties involved is critical for defining the system's functional boundaries [1.3].

STAKEHOLDER	ROLE	KEY REQUIREMENTS
Platform Admin	Owner/Moderator	Commission management, vendor approval, and global sales reports.
Vendors	Independent Sellers	Easy product listing, order management, and payout tracking.
Customers	Buyers	Secure checkout, order history, and real-time tracking.
Stripe	Payment Processor	PCI compliance adherence and API stability [1.4].

3.2 User Stories & Use Cases

These scenarios define the interaction between users and the system [1.3]:

- **Vendor User Story:** "As a Vendor, I want to manage my listings (edit price, update stock) easily so that I can maintain accurate information and increase my sales."
- **Customer User Story:** "As a Customer, I want to track my order in real-time so that I am informed of the delivery status and feel secure in my purchase."
- **Admin Use Case:** Approving a new vendor application to ensure marketplace quality and legal compliance.

3.3 Functional Requirements

The platform must support the following core features:

1. **Product Management:** Vendors can perform CRUD operations on their products including images, descriptions, and categories.
2. **Checkout System:** Integration of a shopping cart with real-time tax and shipping calculations.
3. **Commission Calculation:** Automatic deduction of a 10% platform fee from each transaction before vendor payout.
4. **Order Status Updates:** Automated email notifications for "Shipped," "Out for Delivery," and "Delivered" statuses.

3.4 Non-functional Requirements

Technical constraints that ensure the platform's reliability and user experience [1.8]:

- **Performance:** The **SQL Server** database must be optimized using EF Core Eager Loading and indexed queries to prevent N+1 query issues during high load.
- **Security:** All transactions must be processed via Stripe's hosted fields to ensure the marketplace never touches raw credit card data (PCI Compliance) [1.4].
- **Usability:** The interface must adhere to Web Content Accessibility Guidelines (WCAG) and provide a responsive experience.
- **Scalability:** The **Clean Architecture + ASP.NET Core MVC** pattern separates concerns into distinct layers (Domain, Application, Infrastructure, Presentation), enabling independent scaling of services. **SQL Server** with Entity Framework Core supports horizontal read scaling via read replicas.
- **Reliability:** Implementation of health checks in **Clean Architecture + ASP.NET Core MVC** infrastructure layer to monitor system status and ensure 99.9% uptime. **SQL Server**'s built-in transaction management and EF Core's optimistic concurrency handling safeguard data integrity under concurrent load.